Type

Attribute

Syntax

```
<Path>.MUAnimations.copyFromSimulationObject
```

Example

```
.Models.Model.MyStation._3D.MyAniObj.MUAnimations.copyFromSimulationObject
```

See also

Use as Animation Object

Accessing Self Animations

__Accessing Self Animations

Plant Simulation provides a set of functions for **Self Animations**.

All objects except *Folder*, *Connector*, *Interface*, and *MUs* provide **Self Animations**.

You can buffer the entirety or all animations of a type in a value of data type *any*, for example:

```
var a : any := .Materialflow.PickAndPlace._3D.SelfAnimations
```

SimTalk provides the *attributes* and *methods* listed in the table of contents to the left for self animations in *3D*.

See also

Edit 3D Properties [dialog] > **Self Animation**

_3D.AniRotationAxis [SimTalk]

Sets the rotation axis for the function *_3D.SelfAnimations.scheduleRotation* of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.AniRotationAxis:real[3]
```

Assignment Value

You can assign an *array* containing three values of data type *real*.

The values set the **X**-component, **Y**-component, and **Z**-component of the rotation axis.

Example

```
MyStation._3D.AniRotationAxis := [0,1,0]
// rotates the object around its y-axis
```

See also

Rotation Axis

_3D.SelfAnimations.scheduleRotation [SimTalk]

_3D.AniRotationCenter [SimTalk]

Sets the rotation center for the function *_3D.SelfAnimations.scheduleRotation* of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.AniRotationCenter:length[3]
```

Assignment Value

You can assign an *array* containing three values of data type *length*.

The values set the **X**-position, the **Y**-position, and the **Z**-position of the rotation center around which the object will be rotated.

Example

```
MyStation._3D.AniRotationCenter := [1,0,0]
// rotates the object with an offset of 1,0,0
```

See also

Rotation Center

_3D.SelfAnimations.scheduleRotation [SimTalk]

_3D.AniTranslationDirection [SimTalk]

Sets the translation direction for the poses animation of the object designated by <Path>.

Remarks

Compare *_3D.getObject* for accessing animatable objects.

Type

Attribute

Syntax

```
<Path>._3D.AniTranslationDirection:length[3]
```

Assignment Value

You can assign an *array* containing three values of data type *length*.

The values set the **X**-component, **Y**-component, and **Z**-component of the translation direction.

Example

```
MyStation._3D.getObject(1).AniTranslationDirection := [1,0,0]
```

See also

_3D.getObject [SimTalk]

_3D.Poses.moveToMUAnimationPosition [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.delete [SimTalk]

Deletes the specified **Self Animation** of the object designated by <Path>.

Remarks

You cannot delete generated animations.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.<AnimationPathName>.delete → boolean
<Path>._3D.SelfAnimations.getAnimation(<...>).delete → boolean
```

Return Value

The return value has the data type *boolean*.

true if the animation was deleted successfully.

false if the animation was not deleted.

Example

```
.MaterialFlow.Station._3D.SelfAnimations.Default.delete
// deletes the selft animation named ‘Default’ of the class of the Station
```

See also

_3D.SelfAnimations.getAnimation [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.getTable [SimTalk]

Queries the animation data of all saved animations of the **Self Animation** of the object designated by <Path> and writes it into the specified table.

Remarks

Plant Simulation automatically formats the passed table.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.<AnimationPathName>.getTable(Target:table)
<Path>._3D.SelfAnimations.getAnimation(<...>).getTable(Target:table)
```

Parameter

The parameter *Target* of data type *table* contains the data of each saved animation.

Example

```
// sea stands for self animation
var sea:= Buffer._3D.SelfAnimations
var created : boolean := sea.createAnimationLine("Animation2")
if created // The animation line was created.
   var t: table // Copy the 'Default' line values to the table.
   sea.Default.getTable(t)
   sea.Animation2.setTable(t)
end
```

See also

Edit 3D Properties [dialog] > **Self Animation**

_3D.SelfAnimations.getTable [SimTalk]

_3D.SelfAnimations.setTable [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.setTable [SimTalk]

_3D.SelfAnimations.getAnimation [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.IsCurve [SimTalk]

Returns if the specified **Self Animation** of the object designated by <Path> is of type **Polycurve** (true) or not (false).

Type

Read-only attribute

Syntax

```
<Path>._3D.SelfAnimations.<AnimationPathName>.IsCurve → boolean
<Path>._3D.SelfAnimations.getAnimation(<...>).IsCurve → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
var b : boolean := .MaterialFlow.Station._3D.SelfAnimations.Default.IsCurve
```

See also

_3D.SelfAnimations.getAnimation [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.IsLine [SimTalk]

Returns if the specified **Self Animation** of the object designated by <Path> is of type **Lines** (true) or not (false).

Type

Read-only attribute

Syntax

```
<Path>._3D.SelfAnimations.<AnimationPathName>.IsLine → boolean
<Path>._3D.SelfAnimations.getAnimation(<...>).IsLine → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
var b : boolean := .MaterialFlow.Station._3D.SelfAnimations.Default.IsLine
```

See also

_3D.SelfAnimations.getAnimation [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.IsSpline [SimTalk]

Returns if the specified **Self Animation** of the object designated by <Path> is of type **Spline** (true) or not (false).

Type

Read-only attribute

Syntax

```
<Path>._3D.SelfAnimations.<AnimationPathName>.IsSpline → boolean
<Path>._3D.SelfAnimations.getAnimation(<...>).IsSpline → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
var b : boolean := .MaterialFlow.Station._3D.SelfAnimations.Default.IsSpline
```

See also

_3D.SelfAnimations.getAnimation [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.play [SimTalk]

Schedules the specified saved **Self Animation** of the object designated by <Path> and then plays the scheduled animations.

Remarks

Before executing the method an implicit call of the method `SelfAnimations.reset` takes place.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.<AnimationPathName>.play([Backwards:boolean:=false
]) → time
<Path>._3D.SelfAnimations.getAnimation(<...>).play([Backwards:boolean:=false
]) → time
```

Parameter

The optional parameter *Backwards* of data type *boolean* sets if the animation on the path is to be played backward (`true`) or forward (`false`).

Do not specify the parameter to play the animation on the path forward.

Default Value of the Parameter

The default value is false.

Return Value

The return value has the data type *time*.

It is the time that the specified animation takes.

Example

```
self.~._3D.SelfAnimations.VerticalAni.play
self.~._3D.SelfAnimations.MyAnimationPath.play
```

See also

_3D.SelfAnimations.pause [SimTalk]

_3D.SelfAnimations.playRotation [SimTalk]

_3D.SelfAnimations.scheduleTranslation [SimTalk]

_3D.CameraAnimations.resetAnimation [SimTalk]

_3D.SelfAnimations.scheduleRotation [SimTalk]

_3D.SelfAnimations.scheduleTranslation [SimTalk]

_3D.SelfAnimations.getAnimation [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.schedule [SimTalk]

Schedules the specified saved animation of the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.<AnimationPathName>.schedule([Backwards:boolean:=f
alse])
<Path>._3D.SelfAnimations.getAnimation(<...>).schedule([Backwards:boolean:=f
alse])
```

Parameter

The optional parameter *Backwards* of data type *boolean* sets if the animation on the path is to be played backward (`true`) or forward (`false`).

Do not specify the parameter to play the animation on the path forward.

To run the animation, call the function *_3D.SelfAnimations.playAnimation*.

Default Value of the Parameter

The default value is false.

Example

```
self.~._3D.SelfAnimations.VerticalAni.schedule
self.~._3D.SelfAnimations.playAnimation
```

See also

_3D.SelfAnimations.pause [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.play [SimTalk]

_3D.SelfAnimations.resetAnimation [SimTalk]

_3D.SelfAnimations.scheduleRotation [SimTalk]

_3D.SelfAnimations.getAnimation [SimTalk]

_3D.CameraAnimations.play [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.setTable [SimTalk]

Overwrites the specified **Self Animation** of the object designated by <Path> with the data of the passed table.

Remarks

The table has to be formatted correctly otherwise *Plant Simulation* cancels your changes.

You can query the format of the table with the method *_3D.SelfAnimations.<AnimationPathName>.getTable*.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.<AnimationPathName>.setTable(Source:table)
<Path>._3D.SelfAnimations.getAnimation(<...>).setTable(Source:table)
```

Parameter

The parameter *Source* of data type *table* designates the table which contains the new saved animation of the object.

Example

```
var sea := Buffer._3D.MUAnimations
var created : boolean := sea.createAnimationLine("Animation2")
if created // The animation line was created.
   var t: table // Copy the 'Default' line values to the table.
   sea.Default.getTable(t)
   sea.Animation2.setTable(t)
end
```

See also

Edit 3D Properties [dialog] > **Self Animation**

_3D.SelfAnimations.<AnimationPathName>.getTable [SimTalk]

_3D.SelfAnimations.getAnimation [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.getTable [SimTalk]

_3D.SelfAnimations.AnimationTimeBlock [SimTalk]

Returns the time which the animation of the next block for the object designated by <Path> will most likely require.

Remarks

The time measurement relates to the simulation time.

Type

Read-only attribute

Syntax

```
<Path>._3D.SelfAnimations.AnimationTimeBlock → time
```

Return Value

The return value has the data type *time*.

Example

```
var t : time := self.~._3D.SelfAnimations.AnimationTimeBlock
```

See also

_3D.SelfAnimations.startNextAnimationBlock [SimTalk]

_3D.SelfAnimations.AnimationTimeTotal [SimTalk]

_3D.SelfAnimations.AnimationTimeTotal [SimTalk]

Returns the time which the animation of all blocks for the object designated by <Path> will most likely require.

Remarks

The time measurement relates to the simulation time.

Type

Read-only attribute

Syntax

```
<Path>._3D.SelfAnimations.AnimationTimeTotal → time
```

Return Value

The return value has the data type *time*.

Example

```
var t : time := self.~._3D.SelfAnimations.AnimationTimeTotal
```

See also

_3D.SelfAnimations.AnimationTimeBlock [SimTalk]

_3D.SelfAnimations.createAnimationCurve [SimTalk]

Creates a new saved animation path of type **Polycurve** for the object designated by <Path>.

Remarks

You can set the values of the animation path with the method *_3D.SelfAnimations.setTable*.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.createAnimationCurve(Name:string) → boolean
```

Parameter

The parameter *Name* of data type *string* designates the name of the new animation path to be created.

Return Value

The return value has the data type *boolean*.

true if the animation path was created successfully, i.e., if the name has not been assigned yet.

Example

```
var a: boolean := self.~._3D.SelfAnimations.createAnimationCurve("Curved
Movement")
if not a
   print "The animation curve could not be created."
end
```

See also

Edit 3D Properties [dialog] > **Self Animation**

_3D.SelfAnimations.getTable [SimTalk]

_3D.SelfAnimations.setTable [SimTalk]

_3D.SelfAnimations.createAnimationLine [SimTalk]

_3D.SelfAnimations.createAnimationSpline [SimTalk]

_3D.SelfAnimations.createAnimationLine [SimTalk]

Creates a new saved animation path of type **Lines** for the object designated by <Path>.

Remarks

You can set the values of the animation path with the method *_3D.SelfAnimations.setTable*.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.createAnimationLine(Name:string) → boolean
```

Parameter

The parameter *Name* of data type *string* designates the name of the new animation path to be created.

Return Value

The return value has the data type *boolean*.

true if the animation path was created successfully, i.e., if the name has not been assigned yet.

Example

```
var sea := Buffer._3D.SelfAnimations
```

See also

Edit 3D Properties [dialog] > **Self Animation**

_3D.SelfAnimations.getTable [SimTalk]

_3D.SelfAnimations.setTable [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.setTable [SimTalk]

_3D.SelfAnimations.createAnimationSpline [SimTalk]

Creates a new saved animation path of type **Spline** for the object designated by <Path>.

Remarks

You can set the values of the animation path with the method *_3D.SelfAnimations.setTable*.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.createAnimationSpline(Name:string) → boolean
```

Parameter

The parameter *Name* of data type *string* designates the name of the new animation path to be created.

Return Value

The return value has the data type *boolean*.

true if the animation path was created successfully, i.e., if the name has not been assigned yet.

Example

```
var a: boolean := self.~._3D.SelfAnimations.createAnimationSpline("spline
movement")
if not a
   print "The animation spline could not be created."
end
```

See also

Edit 3D Properties [dialog] >Self Animation

_3D.SelfAnimations.getTable [SimTalk]

_3D.SelfAnimations.setTable [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.setTable [SimTalk]

_3D.SelfAnimations.createAnimationSpline [SimTalk]

_3D.SelfAnimations.deleteAllAnimationBlocks [SimTalk]

Deletes all scheduled animation blocks for the object designated by <Path>.

Remarks

The method does not change any saved animations, but only affects scheduled animations.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.deleteAllAnimationBlocks
```

Example

```
self.~._3D.SelfAnimations.deleteAllAnimationBlocks
```

See also

_3D.SelfAnimations.deleteNextAnimationBlock [SimTalk]

_3D.SelfAnimations.deleteNextAnimationBlock [SimTalk]

Deletes the entire next scheduled animation block for the object designated by <Path>.

Remarks

The method does not change any saved animations, but only affects scheduled animations.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.deleteNextAnimationBlock
```

Example

```
self.~._3D.SelfAnimations.deleteNextAnimationBlock
```

See also

_3D.SelfAnimations.deleteAllAnimationBlocks [SimTalk]

_3D.SelfAnimations.startNextAnimationBlock [SimTalk]

_3D.SelfAnimations.getAnimation [SimTalk]

Returns a **Self Animation** for the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.getAnimation(AnimationPathName:string)
<Path>._3D.SelfAnimations.getAnimation(Index:integer)
```

Parameters

The parameters designate a possibility each to query the designated animation.

- The parameter *AnimationPathName* of data type *string* designates a saved animation, whose name is an illegal *SimTalk* identifier, `"#0#0"` in the example below.
- The parameter *Index* of data type *integer* designates the n-th animation of the animations.

Return Value

Void if you specify a *string* and if the animation with the specified name does not exist. This is instead of opening the *Debugger*.

This does not apply to all other error cases, for example if you only specify a single *integer* value, the direct path extension, or if you specify unsuited data types.

Examples

```
var t: table
self.~._3D.SelfAnimations.getAnimation("#0#0").getTable(t)
```

```
var t: table
self.~._3D.SelfAnimations.getAnimation(7).getTable(t)
```

Accessing an Animation via the Path Extension

If a saved animation has a name that is a legal *SimTalk* identifier, you can access this animation via the path extension.

_3D.SelfAnimations.getTable [SimTalk]

Returns the animation data of all saved animations of the **Self Animation** of the object designated by <Path> and writes it into the specified table.

Remarks

Plant Simulation automatically formats the passed *table*.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.getTable(Target:table)
```

Parameter

The parameter *Target* of data type *table* contains all available saved animations of the designated animation type.

Example

```
var t : table
self.~._3D.SelfAnimations.getTable(t)
debug
self.~._3D.SelfAnimations.setTable(t)
```

See also

_3D.SelfAnimations.setTable [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.getTable [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.setTable [SimTalk]

_3D.SelfAnimations.pause [SimTalk]

Pauses **Self Animations** of the object designated by <Path> which are playing at the moment.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.pause
```

Example

```
self.~._3D.SelfAnimations.pause
```

See also

_3D.SelfAnimations.play [SimTalk]

_3D.SelfAnimations.resetAnimation [SimTalk]

_3D.SelfAnimations.play [SimTalk]

Plays all scheduled **Self Animations** of the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.play
```

Examples

```
var animations : any := self.~._3D.getObject("Arm").SelfAnimations
animations.reset
animations.RotateDown.schedule
animations.startNextAnimationBlock
animations.FinishDown.schedule
animations.scheduleRotation(0, 360, 90)
@.OutIn(animations.AnimationTimeTotal)
animations.play
```

```
self.~._3D.SelfAnimations.scheduleRotation(0, 90, 15)
self.~._3D.SelfAnimations.play
```

See also

_3D.SelfAnimations.pause [SimTalk]

_3D.SelfAnimations.resetAnimation [SimTalk]

_3D.SelfAnimations.scheduleRotation [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.schedule [SimTalk]

Video on YouTube

https://youtu.be/YleFq6afRs4?si=ucjE6lapZOgTy3Tt&t=413

_3D.SelfAnimations.playRotation [SimTalk]

Schedules a rotation animation of the **Self Animation** for the object designated by <Path> and then plays the scheduled animations.

Remarks

The rotation takes place around the **Rotation Axis** and the **Rotation Center** which you defined on the tab **Self Animation**.

> **Note**
> Before the method is being executed an implicit call of the method `_3D.SelfAnimations.reset` takes place.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.playRotation(StartRotationAngle:real,
TargetRotationAngle:real, AngleVelocity:real) → time
```

Instead, you can also use the abbreviated notation:

```
<Path>._3D.playRotation(StartRotationAngle:real, TargetRotationAngle:real,
AngleVelocity:real) → time
```

Parameters

You can specify the following parameters:

- The parameter *StartRotationAngle* of data type *real* designates the start rotation angle in degrees.
- The parameter *TargetRotationAngle* of data type *real* designates the target rotation angle in degrees.
- The parameter *AngleVelocity* of data type *real* designates the angle velocity in degrees per second.

Return Value

The return value has the data type *time*.

It designates the time that the specified translation takes. This way you can directly use the return value as parameter for a *wait instruction* if *Plant Simulation* is to wait for the end of the animation.

Examples

```
self.~._3D.SelfAnimations.playRotation(0, 90, 15)
```

```
self.~._3D.playRotation(0, 90, 15) // abbreviated form
```

```
wait self.~._3D.playRotation(0, 90, 15)
// wait until the rotation is finished
```

See also

_3D.AniRotationAxis [SimTalk]

_3D.AniRotationCenter [SimTalk]

Rotation Axis

Rotation Center

Video on YouTube

https://youtu.be/YleFq6afRs4?si=mgK_XmW4VrwksBEv&t=813

_3D.SelfAnimations.playTranslation [SimTalk]

Schedules an anonymous translation animation of the **Self Animation** for the object designated by <Path> and then plays the scheduled animations.

Remarks

Before executing the method an implicit call of the method `SelfAnimations.reset` takes place.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.playTranslation(StartTranslation:length[3],
TargetTranslation:length[3], Speed:speed) → time
```

Instead, you can also use the abbreviated notation:

```
<Path>._3D.playTranslation(StartTranslation:real[3],
TargetTranslation:real[3], Speed:speed) → time
```

Parameters

You can specify the following parameters:

- The parameter *StartTranslation* is an *array* of data type *length* with three values. They designate the start translation.
- The parameter *TargetTranslation* is an *array* of data type *length* with three values. They designate the target translation.
- The parameter *Speed* of data type *speed* designates the speed of the object in meters per second.

Return Value

The return value has the data type *time*.

It designates the time that the specified translation takes. This way you can directly use the return value as parameter for a *wait-instruction* if *Plant Simulation* is to wait for the end of the animation.

Examples

```
// Moves the existing animation object of the Station with an offset in
// the x direction from 0 to 10 meters. The speed is 2m/s.
```

```
var myAnimationObj := Station._3D.getObject(1)
myAnimationObj.SelfAnimations.playTranslation([0,0,0], [10,0,0], 2.0)

// Instead you can also use the abbreviated notation:
// Moves the existing animation object of the Station with an offset in
// the x direction from 0 to 10 meters. The speed is 2m/s.
var myAnimationObj := Station._3D.getObject(1)
myAnimationObj.playTranslation([0,0,0], [10,0,0], 2.0)
```

```
wait Station._3D.playTranslation([0,0,0], [0,0,3.8], 1.2)
```

```
wait Station._3D.SelfAnimations.playTranslation([0,0,0], [0,0,3.8], 1.2)
```

Video on YouTube

https://youtu.be/YleFq6afRs4?si=zphbmwl6vT5FBJIJ&t=560

_3D.SelfAnimations.resetAnimation [SimTalk]

Resets all scheduled **Self Animations** of the object designated by <Path>.

Remarks

Resetting the animation stops the animation that is played at the moment and cancels all scheduled animations.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.resetAnimation
```

Examples

```
var animations : any := self.~._3D.getObject("Arm").SelfAnimations
animations.resetAnimation
animations.DownAdvance.schedule
animations.startNextAnimationBlock
animations.DownFinal.schedule
animations.scheduleRotation(0, 360, 90)
@.outIn(animations.AnimationTimeTotal)
animations.play
```

```
self.~._3D.SelfAnimations.resetAnimation
```

See also

_3D.SelfAnimations.pause [SimTalk]

_3D.CameraAnimations.play [SimTalk]

_3D.SelfAnimations.scheduleRotation [SimTalk]

Schedules a rotation animation of the **Self Animation** for the object designated by <Path>.

Remarks

The rotation takes place around the **Rotation Axis** and the **Rotation Center** which you defined on the tab **Self Animation**.

> **Note**
> To run the animation, call the function *_3D.SelfAnimations.play*.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.scheduleRotation(StartRotationAngle:real,
TargetRotationAngle:real, AngleVelocity:real)
```

Parameters

You can specify the following parameters:

- The parameter *StartRotationAngle* of data type *real* designates the start rotation angle in degrees.
- The parameter *TargetRotationAngle* of data type *real* designates the target rotation angle in degrees.
- The parameter *AngleVelocity* of data type *real* designates the angle velocity in degrees per second.

Examples

```
var animations : any := self.~._3D.getObject("Arm").SelfAnimations
animations.reset
animations.DownAdvance.schedule
animations.startNextAnimationBlock
animations.DownFinal.schedule
animations.scheduleRotation(0, 360, 90)
@.outIn(animations.AnimationTimeTotal)
animations.play
```

```
self.~._3D.SelfAnimations.scheduleRotation(0, 90, 15)
self.~._3D.SelfAnimations.play
```

See also

_3D.AniRotationAxis [SimTalk]

_3D.AniRotationCenter [SimTalk]

_3D.SelfAnimations.play [SimTalk]

Rotation Axis

Rotation Center

_3D.SelfAnimations.scheduleTranslation [SimTalk]

Schedules an anonymous translation animation of the **Self Animation** for the object designated by <Path>.

Remarks

To run the animation, call the function *_3D.SelfAnimations.play*.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.scheduleTranslation(StartTranslation:length[3],
TargetTranslation:length[3], Speed:speed)
```

Parameters

You can specify the following parameters:

- The parameter *StartTranslation* is an *array* of data type *length* with three values. They designate the start translation.
- The parameter *TargetTranslation* is an *array* of data type *length* with three values. They designate the target translation.
- The parameter *Speed* of data type *speed* designates the speed of the object in meters per second.

Example

```
// Drives the existing animation object of the Station with an offset in
// the x direction from 0 to 10 meters. The speed is 2m/s.
var myAnimationObj := Station._3D.getObject(1)
myAnimationObj.SelfAnimations.reset
myAnimationObj.SelfAnimations.scheduleTranslation([0,0,0], [10,0,0], 2.0)
myAnimationObj.SelfAnimations.play
```

See also

_3D.SelfAnimations.play [SimTalk]

_3D.SelfAnimations.setTable [SimTalk]

Overwrites the saved animations of the designated **Self Animation** of the object designated by <Path> with the data of the passed table.

Remarks

The table has to be formatted correctly otherwise *Plant Simulation* cancels your changes. You can get the format of the table with the method *_3D.SelfAnimations.getTable*.

The animations of the *table*, which the parameter *Source* contains, do overwrite the other previously existing saved animations.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.setTable(Source:table)
```

Parameter

The parameter *Source* of data type *table* contains the new animations.

Example

```
var t: table
self.~._3D.SelfAnimations.getTable(t)
debug
self.~._3D.SelfAnimations.setTable(t)
```

See also

Edit 3D Properties [dialog] > **Self Animation**

_3D.SelfAnimations.getTable [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.getTable [SimTalk]

_3D.SelfAnimations.<AnimationPathName>.setTable [SimTalk]

_3D.SelfAnimations.startNextAnimationBlock [SimTalk]

Terminates the currently playing animation block for the object designated by <Path> and starts the next animation block.

Remarks

3D only plays animations of an animation block. When this block has finished playing, the animation proceeds to the following block. Effects of a block do not have an effect once that block is finished. The animation of the last block remains active until you reset the animation.

Type

Method

Syntax

```
<Path>._3D.SelfAnimations.startNextAnimationBlock
```

Examples

```
self.~._3D.SelfAnimations.VerticalAni.schedule
self.~._3D.SelfAnimations.scheduleRotation(0, 90, 30)
self.~._3D.SelfAnimations.startNextAnimationBlock
self.~._3D.SelfAnimations.HorizontalAni.schedule
self.~._3D.SelfAnimations.scheduleRotation(90, 45, 15)
self.~._3D.SelfAnimations.startNextAnimationBlock
self.~._3D.SelfAnimations.scheduleRotation(45, 0, 10)
self.~._3D.SelfAnimations.play
```

```
var animations : any := self.~._3D.getObject("Arm").SelfAnimations
animations.reset
animations.DownAdvance.schedule
animations.startNextAnimationBlock
animations.DownFinal.schedule
animations.scheduleRotation(0, 360, 90)
@.outIn(animations.AnimationTimeTotal)
animations.play
```

See also

_3D.SelfAnimations.deleteNextAnimationBlock [SimTalk]

_3D.SelfAnimations.AnimationTimeBlock [SimTalk]

Accessing Camera Animations

__Accessing Camera Animations

Plant Simulation provides a set of functions for **Camera Animations**.

Only the *Frame* provides **Camera Animations**.

You can buffer the entirety or all animations of a type in a value of data type *any*, for example:

```
var a : any := .Models.Model._3D.CameraAnimations
```

SimTalk provides the *attributes* and *methods* listed in the table of contents to the left for camera animations in *3D*.

See also

Edit 3D Properties [dialog] > **Camera Animation [Frame]**

_3D.CameraAnimations.<AnimationPathName>.delete [SimTalk]

Deletes the specified **Camera Animation** of the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.<AnimationPathName>.delete → boolean
<Path>._3D.CameraAnimations.getAnimation(<...>).delete → boolean
```

Return Value

The return value has the data type *boolean*.

true if the animation was deleted successfully.

false if the animation was not deleted.

Example

```
_3D.CameraAnimations.Default.delete
```

See also

_3D.CameraAnimations.getAnimation [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.getTable [SimTalk]

Returns the animation data of all saved animations of the specified **Camera Animation** of the object designated by <Path> and writes it into the specified table.

Remarks

Plant Simulation automatically formats the passed table.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.<AnimationPathName>.getTable(Target:table)
<Path>._3D.CameraAnimations.getAnimation(<...>).getTable(Target:table)
```

Parameter

The parameter *Target* of data type *table* contains the data of each saved animation.

Example

```
// cama stands for camera animation
var cama:= _3D.CameraAnimations
var created : boolean := cama.createAnimationLine("Animation2")
if created // The animation line was created.
   var t: table // Copy the ‘Default’ line values to the table.
   cama.Default.getTable(t)
   cama.Animation2.setTable(t)
end
```

See also

Edit 3D Properties [dialog] > **Camera Animation [Frame]**

_3D.CameraAnimations.<AnimationPathName>.setTable [SimTalk]

_3D.CameraAnimations.getTable [SimTalk]

_3D.CameraAnimations.setTable [SimTalk]

_3D.CameraAnimations.getAnimation [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.IsCurve [SimTalk]

Returns if the specified **Camera Animation** of the object designated by <Path> is of type **Polycurve** or not.

Type

Read-only attribute

Syntax

```
<Path>._3D.CameraAnimations.<AnimationPathName>.IsCurve → boolean
<Path>._3D.CameraAnimations.getAnimation(<...>).IsCurve → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
var b : boolean := _3D.MUAnimations.Default.IsCurve
```

See also

_3D.CameraAnimations.getAnimation [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.IsLine [SimTalk]

Returns if the specified **Camera Animation** of the object designated by <Path> is of type **Lines** or not.

Type

Read-only attribute

Syntax

```
<Path>._3D.CameraAnimations.<AnimationPathName>.IsLine → boolean
<Path>._3D.CameraAnimations.getAnimation(<...>).IsLine → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
var b : boolean := _3D.MUAnimations.Default.IsLine
```

See also

_3D.CameraAnimations.getAnimation [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.IsSpline [SimTalk]

Returns if the specified **Camera Animation** of the object designated by <Path> is of type **Spline** or not.

Type

Read-only attribute

Syntax

```
<Path>._3D.CameraAnimations.<AnimationPathName>.IsSpline → boolean
<Path>._3D.CameraAnimations.getAnimation(<...>).IsSpline → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
var b : boolean := _3D.CameraAnimations.Default.IsSpline
```

See also

_3D.CameraAnimations.getAnimation [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.play [SimTalk]

Schedules the saved **Camera Animation** of the *Frame* designated by <Path> and then plays the scheduled animations.

Remarks

Before executing the method an implicit call of the method `CameraAnimations.reset` takes place.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.<AnimationPathName>.play([Backwards:boolean:=false, Factor:real:=1]) → time
<Path>._3D.CameraAnimations.getAnimation(<...>).play([Backwards:boolean:=false]) → time
```

Parameters

You can specify the following parameters:

- The optional parameter *Backwards* of data type *boolean* sets if the animation on the path is to be played backward (`true`) or forward (`false`).

  Do not specify the parameter to play the animation on the path forward.

  Default Value of the Parameter

  The default value is false.

- The optional parameter *Factor* of data type *real* sets a positive factor, which *Plant Simulation* multiplies onto the animation speed.

  Do not specify the parameter to make the animation run as-is, meaning with the factor 1.

  Specify a factor of `0.5` to make the animation run slower.

  Specify a factor of 2 to make the animation run twice as fast, meaning that it takes only half the time.

Return Value

The return value has the data type *time*.

It is the time that the specified animation takes.

Example

```
self.~._3D.CameraAnimations.VerticalAni.play
self.~._3D.CameraAnimations.MyAnimationPath.play
```

See also

_3D.CameraAnimations.pause [SimTalk]

_3D.CameraAnimations.resetAnimation [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.schedule [SimTalk]

_3D.CameraAnimations.getAnimation [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.schedule [SimTalk]

Schedules a saved camera animation of the *Frame* designated by <Path>.

Remarks

To run the animation, call the function *_3D.CameraAnimations.play*.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.<AnimationPathName>.schedule([Backwards:boolean:
=false, Factor:real:=1]])
<Path>._3D.CameraAnimations.getAnimation(<...>).schedule([Backwards:boolean:
=false])
```

Parameters

You can specify the following parameters:

- The optional parameter *Backwards* of data type *boolean* sets if the animation on the path is to be played backward (`true`) or forward (`false`).

  Do not specify the parameter to play the animation on the path forward.

  Default Value of the Parameter

  The default value is false.

- The optional parameter *Factor* of data type *real* sets a positive factor which *Plant Simulation* multiplies onto the animation speed.

  Do not specify the parameter to make the animation run as-is, meaning with the factor 1.

  Specify a factor of `0.5` to make the animation run slower.

  Specify a factor of 2 to make the animation run twice as fast, meaning that it takes only half the time.

Example

```
self.~._3D.CameraAnimations.VerticalAni.schedule
self.~._3D.CameraAnimations.playAnimation
```

See also

_3D.CameraAnimations.pause [SimTalk]

_3D.CameraAnimations.resetAnimation [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.play [SimTalk]

_3D.CameraAnimations.getAnimation [SimTalk]

_3D.CameraAnimations.play [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.setTable [SimTalk]

Overwrites the saved animations of the designated **Camera Animation** data of the *Frame* designated by <Path> with the data of the passed table.

Remarks

The table has to be formatted correctly, otherwise *Plant Simulation* cancels your changes.

You can query the format of the table with the method *_3D.CameraAnimations.<AnimationPathName>.getTable*.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.<AnimationPathName>.setTable(Source:table)
<Path>._3D.CameraAnimations.getAnimation(<...>).setTable(Source:table)
```

Parameter

The parameter *Source* of data type *table* contains the new saved animation.

Example

```
var cama := _3D.CameraAnimations
var created : boolean := cama.createAnimationLine("Animation2")
if created // The animation line was created.
   var t: table // Copy the 'Default' line values to the table.
   cama.Default.getTable(t)
   cama.Animation2.setTable(t)
end
```

See also

Edit 3D Properties [dialog] > **MU Animation [tab]**

_3D.CameraAnimations.<AnimationPathName>.getTable [SimTalk]

_3D.CameraAnimations.getTable [SimTalk]

_3D.CameraAnimations.getAnimation [SimTalk]

_3D.CameraAnimations.AnimationTimeBlock [SimTalk]

Returns the time which the **Camera Animation** of the next block for the *Frame* designated by <Path> will most likely require. The time measurement relates to the simulation time.

Remarks

For **Camera Animations** a block generally only consists of a single animation, overlapping as with **Self Animations** is not possible. The time measurement relates to the simulation time.

Type

Read-only attribute

Syntax

```
<Path>._3D.CameraAnimations.AnimationTimeBlock -> time
```

Return Value

The return value has the data type *time*.

Example

```
var t : time := self.~._3D.CameraAnimations.AnimationTimeBlock
```

See also

_3D.CameraAnimations.AnimationTimeTotal [SimTalk]

_3D.CameraAnimations.AnimationTimeTotal [SimTalk]

Returns the time which the **Camera Animation** of all blocks for the *Frame* designated by <Path> will most likely require.

Remarks

The time measurement relates to the simulation time.

Type

Read-only attribute

Syntax

```
<Path>._3D.CameraAnimations.AnimationTimeTotal -> time
```

Return Value

The return value has the data type *time*.

Example

```
var t : time := self.~._3D.CameraAnimations.AnimationTimeTotal
```

See also

_3D.CameraAnimations.AnimationTimeBlock [SimTalk]

_3D.CameraAnimations.Count [SimTalk]

Returns the number of saved animation paths for the designated **Camera Animation** of the *Frame* designated by <Path>.

Type

Read-only attribute

Syntax

```
<Path>._3D.CameraAnimations.Count → integer
```

Return Value

The return value has the data type *integer*.

Example

```
var cama := MyFrame._3D.CameraAnimations
```

_3D.CameraAnimations.createAnimationCurve [SimTalk]

Creates a new saved animation path of type **Polycurve** for the *Frame* designated by <Path>.

Remarks

You can set the values of the animation path with the method *_3D.CameraAnimations.setTable*.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.createAnimationCurve(Name:string) → boolean
```

Parameter

The parameter *Name* of data type *string* designates the name of the new animation path to be created.

Return Value

The return value has the data type *boolean*.

true if the animation path was created successfully, i.e., if the name has not been assigned yet.

Example

```
var a: boolean := self.~._3D.CameraAnimations.createAnimationCurve("curved
movement")
if not a
   print "The animation curve could not be created."
end
```

See also

Edit 3D Properties [dialog] > **Camera Animation [Frame]**

_3D.CameraAnimations.getTable [SimTalk]

_3D.CameraAnimations.setTable [SimTalk]

_3D.CameraAnimations.createAnimationLine [SimTalk]

_3D.CameraAnimations.createAnimationSpline [SimTalk]

_3D.CameraAnimations.createAnimationLine [SimTalk]

Creates a new saved animation path of type **Lines** for the *Frame* designated by <Path>.

Remarks

You can set the values of the animation path with the method *_3D.CameraAnimations.setTable*.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.createAnimationLine(Name:string) → boolean
```

Parameter

The parameter *Name* of data type *string* designates the name of the new animation path to be created.

Return Value

The return value has the data type *boolean*.

true if the animation path was created successfully, i.e., if the name has not been assigned yet.

Example

```
var a: boolean := self.~._3D.CameraAnimations.createAnimationLine("straight
movement")
if not a
   print "The animation line could not be created."
end
```

See also

Edit 3D Properties [dialog] > **Camera Animation [Frame]**

_3D.CameraAnimations.getTable [SimTalk]

_3D.CameraAnimations.setTable [SimTalk]

_3D.CameraAnimations.createAnimationCurve [SimTalk]

_3D.CameraAnimations.createAnimationSpline [SimTalk]

_3D.CameraAnimations.createAnimationSpline [SimTalk]

Creates a new saved animation path of type **Spline** for the *Frame* designated by <Path>.

Remarks

You can set the values of the animation path with the method *_3D.CameraAnimations.setTable*.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.createAnimationSpline(Name:string) → boolean
```

Parameter

The parameter *Name* of data type *string* designates the name of the new animation path to be created.

Return Value

The return value has the data type *boolean*.

true if the animation path was created successfully, i.e., if the name has not been assigned yet.

Example

```
var a: boolean := self.~._3D.CameraAnimations.createAnimationSpline("spline
movement")
if not a
   print "The animation spline could not be created."
end
```

See also

Edit 3D Properties [dialog] >**Camera Animation [Frame]**

_3D.CameraAnimations.getTable [SimTalk]

_3D.CameraAnimations.setTable [SimTalk]

_3D.CameraAnimations.createAnimationCurve [SimTalk]

_3D.CameraAnimations.createAnimationLine [SimTalk]

_3D.CameraAnimations.getAnimation [SimTalk]

Returns the specified **Camera Animation** of the *Frame* designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.getAnimation(AnimationPathName:string)
<Path>._3D.CameraAnimations.getAnimation(Index:integer)
```

Parameters

The parameters designate a possibility each to query the designated animation.

- The parameter *AnimationPathName* of data type *string* designates a saved animation, whose name is an illegal *SimTalk* identifier, `"#0#0"` in the example below.
- The parameter *Index* of data type *integer* designates the n-th animation of the animations.

Return Value

If a *string* is passed and if the animation with the specified name or the specified index pair does not exist, the method returns void instead of opening the *Debugger*.

This does not apply to all other error cases, for example if you only specify a single *integer* value, the direct path extension, or specifying unsuited data types.

Example

```
var t: table
self.~._3D.CameraAnimations.getAnimation("#0#0").getTable(t)

var t: table
self.~._3D.CameraAnimations.getAnimation(7).getTable(t)
```

See also

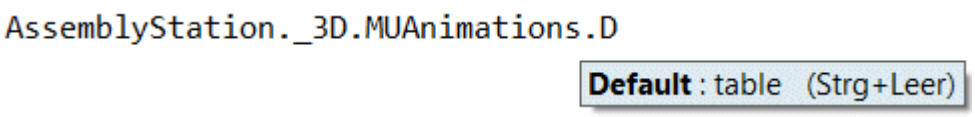
_3D.CameraAnimations.getTable [SimTalk]

_3D.CameraAnimations.setTable [SimTalk]

Edit 3D Properties [dialog] > **Camera Animation [Frame]**

Accessing an Animation via the Path Extension

If a saved animation has a name that is a legal *SimTalk* identifier, you can access this animation via the path extension. This applies, for example, to the **Path Name** > **Default** of the **MU Animation**. Here you could, for example, type in `Assembly._3D.CameraAnimations.D`. The path extension then suggests



_3D.CameraAnimations.getTable [SimTalk]

Returns the animation data of all saved animations of the designated **Camera Animation** of the *Frame* designated by <Path> and writes it into the designated table.

Remarks

Plant Simulation automatically formats the passed *table*.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.getTable(Target:table)
```

Parameter

The parameter *Target* of data type *table* contains all available saved animations of the designated animation type.

Example

```
var t : table
self.~._3D.CameraAnimations.getTable(t)
debug
self.~._3D.CameraAnimations.setTable(t)
```

See also

_3D.CameraAnimations.setTable [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.getTable [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.setTable [SimTalk]

_3D.CameraAnimations.pause [SimTalk]

Pauses animations of the *Frame* designated by <Path> which are playing at the moment.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.pause
```

Example

```
self.~._3D.CameraAnimations.pause
```

See also

_3D.CameraAnimations.play [SimTalk]

_3D.CameraAnimations.resetAnimation [SimTalk]

_3D.CameraAnimations.play [SimTalk]

Plays all scheduled animations of the *Frame* designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.play
```

Examples

```
var animations : any := self.~._3D.getObject("Arm").CameraAnimations
animations.reset
animations.rotateDown.schedule
animations.startNextAnimationBlock
animations.downFinished.schedule
animations.scheduleRotation(0, 360, 90)
@.austrittIn(animations.AnimationTimeTotal)
animations.play
```

```
self.~._3D.CameraAnimations.scheduleRotation(0, 90, 15)
self.~._3D.CameraAnimations.play
```

See also

_3D.CameraAnimations.pause [SimTalk]

_3D.CameraAnimations.resetAnimation [SimTalk]

_3D.CameraAnimations.resetAnimation [SimTalk]

Resets all scheduled **Camera Animations** of the *Frame* designated by <Path>.

Remarks

This stops all playing animations and cancels all scheduled animations.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.resetAnimation
```

Example

```
var animations : any := self.~.Models.MyFrame._3D.CameraAnimations
animations.resetAnimation
animations.Roundtrip.schedule
animations.play

self.~._3D.CameraAnimations.resetAnimation
```

See also

_3D.CameraAnimations.play [SimTalk]

_3D.CameraAnimations.pause [SimTalk]

_3D.CameraAnimations.setTable [SimTalk]

Overwrites the saved animations of the designated **Camera Animation** of the *Frame* designated by <Path> with the data of the passed table.

Remarks

The table has to be formatted correctly, otherwise *Plant Simulation* cancels your changes.

You can get the format of the table with the method *_3D.CameraAnimations.getTable*.

Type

Method

Syntax

```
<Path>._3D.CameraAnimations.setTable(Source:table)
```

Parameter

The parameter *Source* of data type *table* contains the new animations.

Example

```
var t: table
self.~._3D.CameraAnimations.getTable(t)
debug
self.~._3D.CameraAnimations.setTable(t)
```

See also

Edit 3D Properties [dialog] > **Camera Animation [Frame]**

_3D.CameraAnimations.getTable [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.getTable [SimTalk]

_3D.CameraAnimations.<AnimationPathName>.setTable [SimTalk]